

PhysCo: A Benchmark for Cognitive–Motor Coupling in Robotic Manipulation

Version 0.1 | 17 September 2026

Abstract

Successful manipulation requires both a correct decision and a physical effect that realizes it. Separate tests of memory, planning, or motor accuracy cannot determine whether an agent keeps these two aligned when objects move, contact fails, or earlier progress becomes invalid. We specify **PhysCo**, a benchmark design organized into a Cognitive Core, a Motor Core, and a Coupling Suite. The two cores retain five existing task definitions each, covering history-dependent decisions, continuous adjustment, insertion, moving-target grasping, and event response. Twelve proposed coupling cases require decisions to determine physical targets, constraints, or timing, and require observed execution outcomes to change subsequent task state or recovery. The planned evaluation compares two system families: coding agents operating through a fixed, non-learned end-effector harness, and end-to-end vision-language-action or world-action policies producing action chunks. Task outcomes are judged by environment-side conditions rather than self-reported completion. We define outcome, progress, timing, and recovery measurements and reserve result tables for twenty-one candidate configurations. This document is a design-stage technical report; implementation calibration and model evaluation remain outstanding.

Status of this draft. This report specifies ten existing task definitions and twelve proposed coupling cases. It includes a planned evaluation of coding agents and end-to-end VLA/WAM policies. No PhysCo model results are reported: all result cells are intentionally unfilled. Existing task descriptions and demonstrations do not establish that the complete evaluation stack is implemented.

1. Introduction

A robot can select the correct object and still miss its grasp. That mistake becomes a different kind of failure if the system proceeds as though the object had been delivered. In a long task, an inaccurate action may corrupt inventory, invalidate a prerequisite, shift a remembered route index, or cause an order to be closed too early. The decisive question is therefore not only whether a system can decide correctly or move accurately, but whether its subsequent decisions remain grounded in what actually happened.

Recent benchmarks provide increasingly broad manipulation tasks, standardized policy comparisons, and targeted memory evaluations (Chen et al., 2026a; Nasiriany et al., 2026; Zhang et al., 2024; Dai et al., 2026; Chen et al., 2026b). They offer useful foundations for task diversity and reproducibility. For the present project, the remaining design need is more specific: retain the cognitive and motor cases already defined, then construct tasks in which their interaction can be examined through observable events. Merely placing a memory question before a difficult grasp would not meet this objective. The remembered answer must constrain the physical action, and the physical outcome must affect what the system should do next.

PhysCo adopts this bidirectional criterion. Its Cognitive Core contains five tasks involving repeated identities, accumulating orders, rotating coordinate frames, unknown control mappings, and route replay. Its Motor Core contains five tasks involving moving grasps, timed presses, linear and angular adjustment, and insertion. The proposed Coupling Suite combines these requirements through twelve scenarios with effect-dependent progression, resource updates, local repair, or resynchronization. The contribution of this draft is their task and evaluation specification, not a measured improvement in model capability.

The evaluation should accommodate both coding agents and learned policies. Following the model/runtime combinations evaluated by RLE-Bench (RLE-Bench contributors, 2026), a coding agent can observe the robot, reason over its history, and generate calls to a fixed motion interface. We propose to adapt the Inspect EEF interface used in RoboDojo’s GPT-6-Astra report (RoboDojo Team, 2026): no learned VLA is placed in the action path. The second track follows RoboDojo’s policy-serving approach, in which a VLA or WAM returns action chunks through XPolicyLab (XPolicyLab Community, 2026). Both tracks face the same task objective and environment-side completion test, but their interfaces, training access, and timing behavior must be disclosed rather than assumed identical.

This report makes three concrete design contributions. First, it consolidates the current ten core definitions without expanding them into the earlier candidate pool. Second, it specifies twelve coupling cases with observable feedback-dependent decisions and explicit distinctions between intended actions and completed effects. Third, it provides a two-track evaluation protocol and unfilled tables for twenty-one candidate model/runtime configurations. The report is intended to support implementation and discussion: task thresholds that are absent from the source materials remain pending, and no results are inferred from another benchmark’s leaderboard.

Retain the current cores; focus new task design on their interaction.

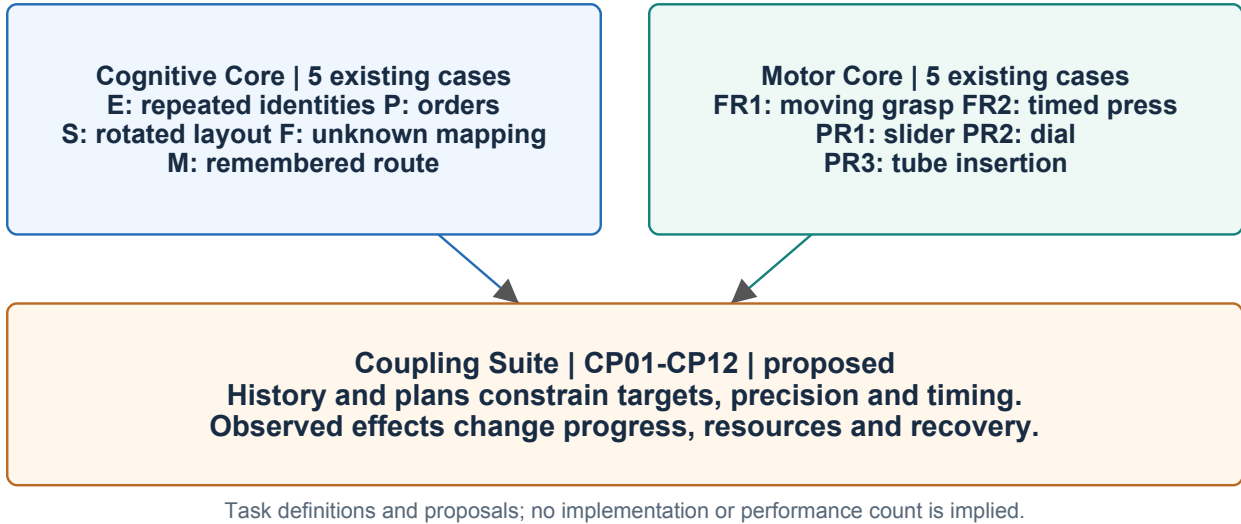


Figure 1: PhysCo task organization. Five cognitive and five motor task definitions are retained. Twelve new coupling cases require task-relevant decisions and observed physical effects to constrain each other. Counts refer to task definitions or proposals, not executed environments or completed experiments.

2. Related Work

Broad manipulation benchmarks. RoboDojo combines 42 simulation tasks with 18 real-world tasks and separates several capability dimensions, including memory, long-horizon execution, and precision (Chen et al., 2026a). We use its technical-report organization and task-card presentation as a reference, while defining a different task set and evaluation objective. RoboTwin 2.0 provides 50 bimanual tasks and structured domain randomization (Chen et al., 2025); RoboCasa365 organizes 365 everyday tasks into atomic and composite behaviors (Nasiriany et al., 2026). VLABench emphasizes semantic understanding, world knowledge, physical reasoning, and long-horizon tasks across 100 categories (Zhang et al., 2024). These resources help identify reusable scenes and operations. Their published task counts and performance values are not PhysCo results, and sharing a primitive does not establish that a proposed coupled task has already been implemented.

Long-horizon execution and memory. RoboCerebra evaluates hierarchical planning and execution over 100 task variants with extended trajectories (Han et al., 2025). RoboMME separates temporal, spatial, object, and procedural memory in sixteen tasks (Dai et al., 2026), while RMBench uses task memory complexity to characterize dependence on historical observations (Chen et al., 2026b). This distinction is central here: a long sequence does not necessarily require hidden historical information, and remembering a fact does not ensure its correct use in control. PhysCo retains explicit history-dependent cases but directs the coupling cases toward target binding, effect confirmation, and state revision. We do not interpret simulation-step count as a memory-capacity measurement.

Agents as robotics problem solvers. RLE-Bench evaluates coding agents across interactive control, policy development, perception, and mechanical design (RLE-Bench contributors, 2026). Its unit of comparison is a model operating inside an agent runtime, with tools and resource budgets. PhysCo borrows the candidate model/runtime identities and the practice of recording interfaces and costs, but does not add those engineering workflows to its task list. The primary evaluation here concerns execution within an episode; development or cross-episode learning belongs to a separate, explicitly declared adaptation condition.

RoboDojo’s GPT-6-Astra report makes a complementary distinction between a learned policy, a planner acting through a learned policy, and a model acting through a non-learned motion harness (RoboDojo Team, 2026). Its Inspect EEF configuration exposes RGB observations, proprioception, and a bounded end-effector tool surface, without a learned VLA or pick/place macros. Inspect Robots provides a broader evaluation framework for tool-driven and learned robot policies (Inspect Robots contributors, 2026). PhysCo adopts the fixed-harness interface boundary for its coding-agent track, and treats the bridge from Codex, Claude Code, or Antigravity CLI to that interface as proposed integration work.

Precision and dynamic execution. Simulation toolkits such as ManiSkill3 support diverse contact and manipulation environments and efficient parallel evaluation (Tao et al., 2024). RoboChallenge provides a real-robot evaluation reference with thirty tabletop tasks (Yakefu et al., 2025). These efforts motivate explicit physical completion conditions and reproducible execution. For PhysCo, a separate timing issue is essential: fast simulated stepping or a 25 Hz interpolated actuator stream does not imply that an agent updates its decisions at that rate. Dynamic cases must disclose whether the world advances

Table 1: The ten existing core definitions. These rows preserve the current task mechanisms. They are not ten newly implemented environments.

ID	Case	Main requirement / important boundary
Cognitive Core		
E	Repeated-item recognition	Retain identities across disappearing batches; grasp after the tray stops.
P	Incremental meal orders	Maintain cumulative demand separately from deposited contents.
S	Color-layout restoration	Track a hidden tray-relative mapping; place after rotation stops.
F	Unknown button mapping	Infer valid press effects and navigate; mapping stays fixed within the episode.
M	Route replay	Track a covered workpiece and reproduce its route, including repeated stations.
Motor Core		
FR1	Conveyor grasping	Intercept a moving box and stably deposit it.
FR2	Illuminated-button response	Reach, press and release for ten successive timed prompts.
PR1	Linear resistance adjustment	Meet a resistance tolerance and hold for one second after release.
PR2	Sequential voltage-dial setting	Meet sequential angular targets with reversals; no backlash.
PR3	Sample-tube insertion	Align with a marked hole and maintain alignment along the guide.

during inference and measure deadlines against the declared environment clock.

3. Benchmark Design and Existing Cores

3.1. Scope and source of task definitions

The current specification contains five Cognitive Core cases, five Motor Core cases, and twelve proposed coupling cases. The core descriptions follow the project’s 17 September 2026 MEMbench source snapshot; the coupling descriptions follow CP01–CP12 from the same day’s design document. We retain these identifiers throughout. Difficulty settings, scene randomizations, robot embodiments, and evaluation episodes do not create additional task families. In particular, this report does not incorporate the earlier 42-family exploration pool.

Each task is described by an instruction, initial conditions, observable information, admissible actions, completion and failure conditions, and a budget. Environment-side state may be used for scoring without being exposed to the policy. A task description is an intended specification; implementation status, robot configuration, and calibrated thresholds are recorded separately. [Table 1](#) summarizes the existing cases, and [section A](#) gives their instructions and known difficulty settings.

3.2. Cognitive Core: retained evidence that changes actions

The cognitive cases emphasize information that is absent from the current image or must be interpreted through earlier interaction. E requires a set of previously seen identities, whereas P requires two evolving quantities: what was requested and what was delivered. S requires a coordinate transformation applied to a remembered mapping. F requires learning a button-direction mapping from actual interventions, then applying it after a reset. M requires an ordered route with repeated visits, not only the target’s final location.

The core is therefore intentionally narrower than a complete test of general reasoning. We retain its existing mechanisms rather than claiming that five cases span all planning, diagnosis, or knowledge-transfer abilities. A model may fail because it lost history, bound the wrong entity, misread the current scene, or failed to execute the selected action. These possibilities motivate both event-level diagnostics and the coupling cases; a low final success rate alone cannot separate them.

3.3. Motor Core: effect, stability and timing

Precision is assessed through task-specific physical conditions. PR1 distinguishes an intended target from the reading after release; its stated resistance tolerances are ± 1 , ± 0.5 , and ± 0.2 ohm across the three levels. PR2 holds appearance, friction and damping fixed and increases the number of sequential targets and direction reversals. Its numeric tolerance remains to be calibrated, and the source explicitly excludes backlash. PR3 varies clearance and effective guide length while retaining one tube and one hole.

The frequency cases test moving-target interception and repeated responses to visual events. In FR2, release must occur within one second of an effective press, and a new prompt arrives 0.3 seconds after release confirmation. These rules do not determine the missing response-deadline values. In both FR1 and FR2, the distinction between command duration, physical motion rate, observation refresh and decision latency must remain visible in the protocol.

3.4. Difficulty and observability

The available source text contains explicit three-level settings for F, M, PR1, PR2 and PR3. Embedded tables for E, P, S, FR1 and FR2 were not retrieved; their numeric or structural level settings remain pending. We do not fill those gaps with thresholds from the coupling proposals. Where a source provides only a qualitative clearance or tolerance, the implementation must calibrate and freeze a numeric range before comparative evaluation.

For a main-track evaluation, both system families receive RGB observations, robot proprioception and the task instruction, with task cues rendered in the scene or supplied through a declared instruction channel. Exact object poses, hidden identities, future events and success-checker internals are reserved for the evaluator. Additional force, audio, metric depth or symbolic event channels define separately labeled conditions. This is particularly relevant to contact cleaning and rhythmic tapping: their main-track realizations must supply enough visible evidence to make the task observable without silently adding a privileged sensor.

4. Coupling Suite

4.1. A bidirectional task criterion

A coupling case must satisfy two dependencies. First, task understanding, history or progress determines a physical target, constraint, order, or opportunity window. Second, the actual physical effect changes a subsequent decision: whether a stage is complete, which item remains pending, which resource is available, or which prerequisite must be rechecked. A task in which reasoning ends before an independent controller solves the remaining problem does not isolate this interaction.

A useful conceptual description separates the task state z_k , the intended operation g_k , and the observed effect y_k :

$$g_k = G(z_k, o_k), \quad y_k = \text{Execute}(g_k), \quad z_{k+1} = U(z_k, o_{k+1}, y_k). \quad (1)$$

Here k indexes decision events, not fixed-frequency control steps. This is a description of task dependencies, not a required policy architecture. An end-to-end policy need not expose z_k or g_k , and a coding agent need not implement named modules G and U . Evaluation instead checks whether its actions and subsequent behavior are consistent with the effects and constraints observable to it.

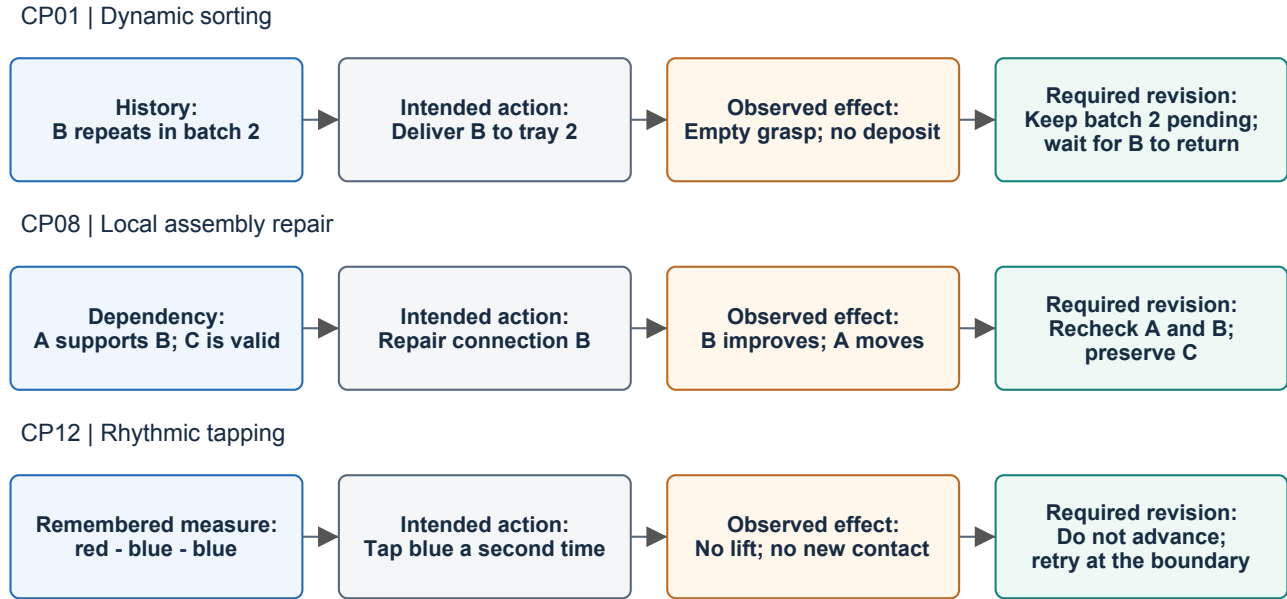
Table 2: Twelve proposed coupling cases. M: historical memory; H: long-horizon structure; P: physical precision; D: dynamic/timing demands. Tags state design intent. CP09 requires actual pour-stop or disturbance-correction timing; waiting alone does not satisfy D.

ID	Proposed case	M	H	P	D
CP01	History-conditioned dynamic sorting	•	—	—	•
CP02	Incremental orders with moving-bin portioning	•	•	•	•
CP03	Remembered-layout restoration on a moving tray	•	—	•	•
CP04	Timed calibration with unknown controls	•	—	•	•
CP05	Remembered routes with timed station handovers	•	•	•	•
CP06	Recipe-driven multistage precision adjustment	•	•	•	—
CP07	Resource-constrained moving-part assembly	•	•	•	•
CP08	Inspection-driven assembly and local repair	—	•	•	—
CP09	Concurrent preparation and quantitative delivery	•	•	•	•
CP10	Precision packing with shared-space constraints	—	•	•	—
CP11	Interlocked cleaning with contact maintenance	—	•	•	•
CP12	Demonstrated rhythmic tapping with resynchronization	•	•	•	•

The twelve cases in [table 2](#) are designs, not reported deployments. Their component sources support particular operations or scene concepts; they do not certify the complete coupled scenario. Each proposed task card in [section B](#) specifies the observation boundary, intended physical interaction, feedback-dependent continuation, success conditions, difficulty progression and reusable components. Easy-to-Medium changes generally increase a cognitive dependency; Medium-to-Hard retains that structure while tightening one physical constraint. When the earlier proposal offered alternative Hard variants, this draft selects one and records it in the English task card rather than changing several physical factors together.

4.2. Three illustrative feedback-dependent decisions

CP01: retain a pending obligation after a miss. An initial batch contains A and B. A later batch contains B and a new item C, so B is the correct target. If the grasp misses, remembering B is not enough: the system must keep the second batch unresolved and use the permitted return window. It must also preserve B’s batch-to-tray binding when the next batch arrives. An observed stable deposit, rather than a completed grasp command, closes the pending item.



Illustrative design episodes, not model rollouts or experimental results.

Figure 2: Designed failure-to-recovery examples. Correctly chosen actions can fail physically; the next decision must then reflect the observed outcome. These are schematic episodes specified for CP01, CP08 and CP12, not recorded model rollouts.

CP08: repair changes what remains certified. Consider an assembly in which A supports B and C is independent. A partial insertion at B requires local repair. If that repair disturbs A, A’s earlier verification becomes invalid even when B now appears correct. A competent continuation rechecks the affected dependency while preserving C. This exposes both overconfident progression and unnecessary global restart; legitimate repair plans may differ.

CP12: contact events determine sequence progress. A demonstrated measure contains red–blue–blue. The second blue requires a new strike after a lift. If the stick remains pressed, the issued command is not a second contact. The system must detect the missed event and resynchronize at the permitted measure boundary, rather than advance its note index and shift the rest of the sequence. A public beat provides current timing but never reveals the target key sequence.

4.3. Feedback without privileged answers

The task cards sometimes refer to a deposit confirmation, inspection indication, quantity reading, or contact event. In the RGB-plus-proprioception main track, these must correspond to evidence a policy can legitimately observe, such as a visible deposited object, a readable scale, a stage lamp or a lifted tapping stick. Evaluator-only truth is not automatically returned as a symbolic message. If an instrumented diagnostic exposes a symbolic event channel, that channel is documented, shared across compared systems and reported separately.

This separation is important for interpreting failures. An evaluator may know that a pin was not inserted, while the model’s image is inconclusive. The appropriate diagnosis may be insufficient observation or unresolved attribution, not failure to use feedback it never received. Likewise, a low-level interlock or motion limiter must be logged when it changes or rejects a request; hidden automatic task completion or replanning would change what the benchmark measures.

5. Evaluation Systems and Interfaces

5.1. Coding agents through a fixed motion harness

The first track evaluates coding agents using the model/runtime combinations listed by RLE-Bench, rather than introducing an additional Planner-plus-VLA family. Table 3 freezes nine candidate combinations from the official public data snapshot generated on 14 September 2026. Names identify external candidates, not API availability or completed PhysCo runs. Exact provider model IDs, runtime versions and inference settings must be recorded when integration is performed.

The proposed robot interface follows the *Inspect EEf* configuration described in RoboDojo’s GPT-6-Astra report (RoboDojo Team, 2026). That report’s instance supplies three RGB cameras, 14-dimensional proprioception and the official task instruction; it exposes end-effector motion and giving up, with no success-declaration tool. Local code bounds a requested target, converts it to a joint trajectory and executes an interpolated chunk. It does not place a learned VLA or a pick/place

Table 3: Coding-agent candidates. Model/runtime names follow RLE-Bench; the PhysCo robot-interface bridge is proposed.

Model	Agent runtime
GPT-6 Astra	Codex
Claude Opus 5	Claude Code
GPT-5.6 Sol	Codex
DeepSeek-V4.1-Flash	Claude Code
Gemini 3.7 Flash	Antigravity CLI
GLM-5.3 Flash	Claude Code
Claude Opus 4.8	Claude Code
GPT-5.6 Luna	Codex
GPT-5.6 Terra	Codex

macro in the action path. The original report uses 25 Hz interpolation on its bimanual tabletop embodiment; this is not a measured PhysCo rate.

For PhysCo, the intended bridge is *coding-agent runtime* $\rightarrow$ *registered Inspect EEF operations* $\rightarrow$ *bounded robot commands*. The agent may use its normal code workspace to organize calculations and task state, but all physical effects must pass through the declared tool surface. Code must not obtain simulator internals, modify the reward, or bypass the motion boundary. The bridge, embodiment-specific proprioception, exact tool schema, observation history policy and runtime restrictions are integration items, not existing capabilities asserted by this report. In particular, the report’s GPT-6 planner invocation is not evidence that the identical harness already runs inside Codex, Claude Code or Antigravity CLI.

5.2. End-to-end VLA and WAM policies

The second track uses learned policies that map instruction and observation history into action chunks. We select twelve initial candidates from RoboDojo’s public policy comparisons, including generalist VLA policies, a memory-oriented VLA, and world-action policies. Table 4 records their names without importing their RoboDojo scores. Older and updated versions are distinct configurations: for example, GalaxeaVLA G0.5 and Xiaomi-Robotics-1 are not aliases for the earlier G0 and Robotics-0 rows.

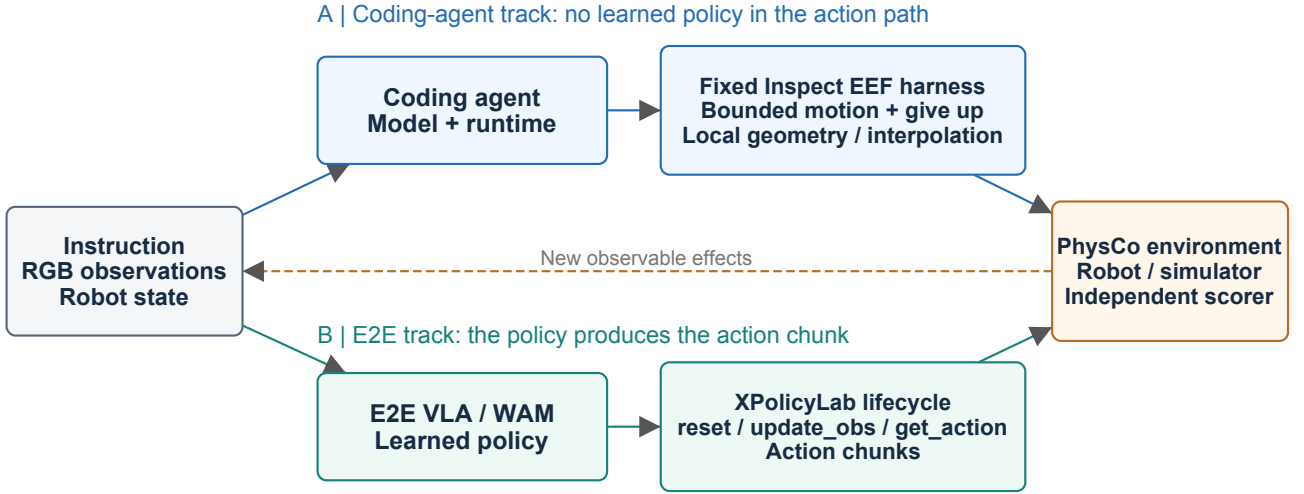
Table 4: E2E candidate policies. Names follow the RoboDojo public comparison. Checkpoints and PhysCo training settings remain pending.

Policy	Selection role
$\pi_{0.5}$	Generalist action policy
OpenVLA-OFT	Fine-tuned VLA reference
GR00T-N1.7	Generalist action policy
Hy-Embodied-0.5-VLA	Broad RoboDojo policy reference
EventVLA	History-oriented policy reference
X-VLA	Generalist action policy
DM0.5	Updated public-board policy
GalaxeaVLA G0.5	Updated Galaxea policy
Xiaomi-Robotics-1	Updated Xiaomi policy
OpenWAM- α	World-action policy reference
X-WAM	World-action policy reference
Fast-WAM	World-action policy reference

XPolicyLab supplies the reference lifecycle: `reset()`, `update_obs(obs)` and `get_action()`, with optional batching and a websocket policy-server boundary (XPolicyLab Community, 2026). In a PhysCo adapter, the instruction, camera ordering, proprioception, units, joint/end-effector action mode, chunk duration and termination behavior must be explicit. Model state is reset between episodes. Checkpoint identity and any PhysCo training or fine-tuning data are part of a run’s provenance, not properties inferred from the model name.

5.3. What a comparison means

The main comparison is between complete systems on common task conditions. It is not automatically a comparison of equal pretraining, parameter count or compute. A coding agent with an untrained motion harness and a fine-tuned VLA have different learning histories. A bounded Cartesian tool and joint-space action chunks also provide different execution support. Tables therefore accompany model names with the available observation, action interface, training/adaptation condition and budget. Results from privileged poses, additional sensors, extra development episodes or a different environment clock form



PhysCo adapters are proposed. Ground-truth scoring state is not returned to the model.

Figure 3: Two planned evaluation tracks. Coding agents act through a fixed non-learned end-effector harness. E2E VLA/WAM policies emit action chunks through the XPolicyLab lifecycle. Both use environment-side outcomes; tool and policy interfaces are disclosed rather than treated as identical. PhysCo adapters remain proposed.

separate labeled conditions.

The primary design starts each evaluation episode with a fresh runtime or policy state and no target-task cross-episode experience. History within the episode is allowed and necessary for the cognitive tasks. A future adaptation track may permit controlled development episodes and freeze the resulting artifacts before held-out evaluation; that extension does not change the baseline task count and is outside the unfilled primary table.

6. Evaluation Protocol and Measurements

6.1. Episodes, budgets and clocks

An evaluation instance fixes the task definition, difficulty, initial condition, external-event seed, observation permissions and declared budgets. The two tracks share the same task-level conditions and success checker. Their internal controller behavior and the resulting future states may differ; using the same initial seed does not make divergent trajectories identical counterfactuals.

Resource limits include elapsed environment time, actuator steps, model/tool calls and any allowed recovery or probe count. The report leaves their uncalibrated values pending. They must be fixed before comparing systems, with aborted and unsuccessful episodes retained. A timeout after a valid episode starts is a task failure. An unavailable model or a failed setup before an episode starts is an integration/coverage status, not a zero success rate over imagined trials.

Dynamic tasks require an explicit clock contract. Under a *continuous-clock* condition, the environment and relevant external events advance while a model reasons or waits for an API response. The physical backend follows a declared hold policy when no new command is ready. Under a *paused-inference* condition, that delay does not advance simulated time. The latter can diagnose task reasoning but does not support a real-time-control claim. These conditions must be recorded and tabulated separately; no effective action frequency is inferred from simulator throughput or interpolated controller ticks.

At minimum, logs retain observation capture, inference start/end, tool dispatch, action acceptance/start/end, feedback capture/delivery and episode termination timestamps. A command identifier links an effect observation to the operation it may confirm. The implementation must document clock alignment, camera age and motion-queue semantics; these details matter directly for CP01, CP03, CP05 and CP12.

6.2. Outcome and progress

For task t , with N_t valid started evaluation episodes and independent environment-side success labels $s_{t,i}$,

$$SR_t = \frac{1}{N_t} \sum_{i=1}^{N_t} s_{t,i}, \quad SR_S = \frac{1}{|S|} \sum_{t \in S} SR_t, \quad SR_{\text{macro}} = \frac{SR_C + SR_M + SR_{CP}}{3}. \quad (2)$$

Here C , M and CP denote the three suites of 5, 5 and 12 definitions. Task-level results are first computed for a fixed difficulty and protocol; any subsequent aggregation over difficulty uses a common prespecified weighting. A suite or overall aggregate

stays unfilled if its required coverage is incomplete. Coupling tasks are not counted multiple times because they carry several capability tags.

Progress is scored using task-defined checkpoints with independent conditions. A checkpoint is credited when its effect is verified and still valid under the task’s rules. A later disturbance may revoke a prerequisite, while a completed route visit remains a historical event. Thus progress is not simply the number of commands sent or the best stage count ever observed. The default proposed coupling progress is the fraction of valid required checkpoints at termination; each task must publish how dependencies, repeated events and revocation work before that score is used.

6.3. Diagnostic measurements

Premature advancement and state mismatch. Premature advancement occurs when behavior commits to a dependent next step before the current physical condition is met. A state-synchronization error occurs when an observable continuation uses a stale or incorrect task state, such as treating a missed deposit as delivered. These are behavioral definitions, not a requirement to inspect a model’s private reasoning. Some systems or episodes will not expose enough evidence for a subtype label; such cases remain unclassified rather than being forced into a category.

Binding, physical residual and recovery. Wrong-target binding includes using the wrong object, container, route node or frame despite a valid physical execution. Residuals are task-specific: position or angle for adjustment, insertion depth for alignment, quantity for delivery, and contact time for tapping. Recovery cost records the additional actions, elapsed time and resources after a detected deviation. A common cost definition is used only where the relevant resource is comparable; resistance error, dose error and route edit distance are not averaged into an unexplained scalar.

Latency and update rate. Decision latency is measured from availability of an input observation to its corresponding command being ready; capture-to-effect delay is separately recoverable from the timestamp trace. Effective action update rate counts independently produced accepted policy/tool commands over active environment time, not the interpolation points within a chunk. Deadline misses use task-defined windows. Report the clock condition, action-chunk duration, API calls and cost alongside latency; model API cost excludes simulator and infrastructure cost unless explicitly included.

6.4. Matched diagnostic interventions

For systems with legally replaceable interfaces, four conditions can distinguish decision and execution limits: Reason-Only uses the native decision component with an oracle executor; Act-Only uses oracle subgoals with native execution; Coupled evaluates the native system; Certified uses oracle decision and execution to check task feasibility. Oracle execution retains action durations and external events. Oracle subgoals cannot leak future arrivals or hidden results. Certified is a feasibility reference, not a model score.

These substitutions are optional diagnostics, not mandatory rows for every policy. In particular, an inseparable E2E model cannot be assigned an invented Reason-Only result. For either track, feasible observation or feedback interventions may instead delay a declared channel, withhold a visible confirmation, or compare effect-confirmed progression with command-based progression. A manipulation must specify exactly what changed, preserve relevant inspection actions and costs, and leave the independent success condition unchanged. Natural divergence of subsequent physical states prevents interpreting a raw success-rate difference as a pure causal interface effect.

6.5. Calibration and held-out conditions

Before producing a leaderboard, each task needs a feasibility check, a frozen initial-state distribution and calibrated tolerances. Difficulty changes must be attributable: keep cognitive demand fixed while varying a physical parameter, or keep physical demand fixed while varying history or dependency structure. A recovery scenario must be possible within the declared budget and expose sufficient evidence of the deviation. Numeric thresholds, episode counts, seeds and uncertainty-reporting choices are recorded as pending in this draft, to be frozen before evaluation rather than selected after observing model rankings.

7. Reserved Results and Implementation Status

7.1. Main comparison

Table 5 reserves rows for the twenty-one candidate configurations. Every measurement is intentionally unfilled. The symbol — means *not evaluated*; it is neither a zero nor an estimate. The model names and runtimes are source-grounded metadata, while PhysCo scores will require new runs. No ranking, best-result highlighting, performance claim or statistical conclusion can be drawn from this table.

The appendix provides the corresponding per-case matrices for all ten core definitions and CP01–CP12, followed by diagnostic and runtime tables. Separate tables should be instantiated for different observation or clock conditions rather than combining incompatible runs. The machine-readable result template represents missing values as `null`; none of the suite aggregates is

Table 5: Reserved PhysCo main comparison. All columns require new PhysCo runs; groups identify distinct action interfaces. All measurement cells are *not evaluated*.

Model / system	Cognitive SR	Motor SR	Coupling SR	Suite macro SR	Coupling progress
Coding agents + proposed Inspect EEF adapter					
GPT-6 Astra	—	—	—	—	—
Claude Opus 5	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—
End-to-end VLA / WAM policies					
$\pi_{0.5}$	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—
EventVLA	—	—	—	—	—
X-VLA	—	—	—	—	—
DM0.5	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—
OpenWAM- α	—	—	—	—	—
X-WAM	—	—	—	—	—
Fast-WAM	—	—	—	—	—

computed from the empty cells. A genuinely inapplicable measurement will be labeled N/A with a reason when configuration details are known.

7.2. What is available and what remains

The report package provides task descriptions, preserved source snapshots, model identities, planned interface boundaries, diagram sources and editable table templates. The current core source includes demonstration references and several difficulty tables, but does not establish a complete evaluation backend. The twelve coupling cases require their additional mechanisms and checks to be implemented. Neither the coding-runtime bridge nor the PhysCo policy-server client is delivered as working evaluation software in this report.

The next validation sequence is to check task observability and oracle feasibility, calibrate physical tolerances, verify adapters and reset behavior, then run matched episodes under a frozen protocol. Early pilots should specifically test that CP01 retains undelivered obligations, CP08 invalidates disturbed prerequisites and CP12 distinguishes a new strike from continued contact. These are acceptance questions for the task implementation, not forecasts of how any candidate model will perform.

7.3. Planned analyses

Once results exist, the primary analysis compares suite completion and task-level variation across coding-agent and E2E configurations. Event traces can then examine whether failures concentrate at target selection, action realization, effect recognition or subsequent state update. The analysis must also report unclassified cases and interface differences. A complementary study can hold the physical task fixed while changing feedback availability, and a timing study can compare continuous-clock and paused-inference conditions. Claims of memory benefit, recovery ability or real-time competence require the corresponding controlled evidence; a stronger aggregate score alone does not establish them.

8. Limitations and Conclusion

This draft defines a research instrument rather than demonstrating its empirical validity. Several original difficulty tables are unavailable, physical thresholds require calibration, and simulation assets from different projects are not interchangeable without implementation work. An interface adaptation may introduce important differences in motion support, observation history and latency. These must be measured and disclosed before interpreting cross-system comparisons. Force, fluids, electrical connection and continuous-clock behavior require particular care: a geometric or discrete proxy is not evidence of the corresponding real-world phenomenon.

The core cases also do not exhaust cognition or motor control, and the twelve coupling proposals are not twelve independent proofs of a single latent capability. Their value depends on distinguishable mechanisms, reliable completion checks, observable feedback and controlled difficulty. Future additions should be motivated by a missing failure mode rather than by a larger count.

PHYSCo organizes the current five cognitive and five motor definitions around twelve proposed tests of cognitive–motor coupling. The central requirement is that task-relevant intent constrains physical action and actual effects constrain subsequent decisions. The two planned model tracks make this requirement accessible to coding agents with a non-learned motion harness and to end-to-end VLA/WAM policies. The present deliverable provides the task, interface and reporting structure needed to implement and evaluate that design; all performance claims remain open.

References

- Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan-ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation, 2025. URL <https://arxiv.org/abs/2506.18088v2>. Downloaded arXiv version 2506.18088v2.
- Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, Honghao Su, Zhiyang Dou, Kaixuan Wang, Dandan Zhang, Yunze Liu, Yan Qin, Qiwei Liang, Qiwei Wu, Zijian Lin, Wenwei Lin, Yuran Wang, Minghua He, Tianshu Wu, Ruihai Wu, Jingquan Zhou, Kai-Chong Lei, Haibao Yu, Yuanfeng Ji, Weiyang Jin, Guanyu Lin, Xiaofan Li, Qi Xiong, Renjing Xu, Zhongyu Li, Wenhao Chai, Enze Xie, Ziwei Wang, Yao Mu, Hao Dong, Wojciech Matusik, Mingyu Ding, Wenbo Ding, Ping Luo, and Masayoshi Tomizuka. Robodojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies, 2026a. URL <https://arxiv.org/abs/2607.04434v3>. Downloaded arXiv version 2607.04434v3.
- Tianxing Chen, Yuran Wang, Mingleyang Li, Yan Qin, Hao Shi, Zixuan Li, Yifan Hu, Yingsheng Zhang, Kaixuan Wang, Yue Chen, Hongcheng Wang, Junjie Wang, Tianhang Yang, Renjing Xu, Ruihai Wu, Yao Mu, Yaodong Yang, Hao Dong, and Ping Luo. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design, 2026b. URL <https://arxiv.org/abs/2603.01229v3>. Downloaded arXiv version 2603.01229v3.
- Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies, 2026. URL <https://arxiv.org/abs/2603.04639v3>. Downloaded arXiv version 2603.04639v3.
- Songhao Han, Boxiang Qiu, Yue Liao, Siyuan Huang, Chen Gao, Shuicheng Yan, and Si Liu. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation, 2025. URL <https://arxiv.org/abs/2506.06677v2>. Downloaded arXiv version 2506.06677v2.
- Inspect Robots contributors. Inspect Robots: An open-source evaluation framework for physical AI. Software repository, 2026. URL <https://github.com/robocurve/inspect-robots>. Accessed 17 September 2026.
- Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots, 2026. URL <https://arxiv.org/abs/2603.04356v1>. Downloaded arXiv version 2603.04356v1.
- RLE-Bench contributors. RLE-Bench: A qualifying exam for coding agents as robot learning engineers. Project repository and public model/runtime data, 2026. URL <https://github.com/RLE-Bench/RLE-Bench>. Accessed 17 September 2026; model data generated 14 September 2026.
- RoboDojo Team. RoboDojo GPT-6-Astra evaluation report. Online evaluation report, including RoboProbe L3 / Inspect EEF, 2026. URL <https://robodojo-benchmark.com/report/gpt-6-astra-eval>. Accessed 17 September 2026; cited as a web report, not a separate peer-reviewed paper.
- Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai, 2024. URL <https://arxiv.org/abs/2410.00425v2>. Downloaded arXiv version 2410.00425v2.
- XPolicyLab Community. XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment. Software repository and adapter documentation, 2026. URL <https://github.com/XPolicyLab/XPolicyLab>. Accessed 17 September 2026; software citation for the interface documentation.
- Adina Yakefu, Bin Xie, Chongyang Xu, Enwen Zhang, Erjin Zhou, Fan Jia, Haitao Yang, Haoqiang Fan, Haowei Zhang, Hongyang Peng, Jing Tan, Junwen Huang, Kai Liu, Kaixin Liu, Kefan Gu, Qinglun Zhang, Ruitao Zhang, Saike Huang, Shen Cheng, Shuaicheng Liu, Tiancai Wang, Tiezhen Wang, Wei Sun, Wenbin Tang, Yajun Wei, Yang Chen, Youqiang Gui, Yucheng Zhao, Yunchao Ma, Yunfei Wei, Yunhuan Yang, Yutong Guo, Ze Chen, Zhengyuan Du, Ziheng Zhang, Ziming Liu, and Ziwei Yan. Robochallenge: Large-scale real-robot evaluation of embodied policies, 2025. URL <https://arxiv.org/abs/2510.17950v1>. Downloaded arXiv version 2510.17950v1.
- Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks, 2024. URL <https://arxiv.org/abs/2412.18194v1>. Downloaded arXiv version 2412.18194v1.

A. Existing Core Task Cards

The following English cards preserve the current ten definitions. Success and diagnostic descriptions specify intended checks; they do not assert completed implementation. Missing original difficulty tables remain unfilled. Source: the preserved MEMbench snapshot dated 17 September 2026.

A.1. E – Repeated-item recognition across batches

Suite. Cognitive Core; existing definition.

Instruction. Observe the batches of objects arriving on the conveyor and remember which objects have appeared. Only observe the first batch. From the second batch onward, exactly one item in each batch has appeared in an earlier batch; all others are new. After the tray stops, identify the repeated item and place it in that batch’s submission tray. Both trays then leave view before the next batch enters. Continue until all batches have been processed.

Task requirement. Identity memory must survive the removal of earlier batches. The target is determined by the union of earlier observations, not only by the immediately preceding batch.

Observation boundary. Current RGB observations, the task instruction and robot state. Previous objects disappear with their trays. Object identities and the repeated-item label are reserved for evaluation.

Completion condition. Every required batch contains the correct submitted repeat item, with no wrong submission. Picking begins after the tray has stopped; this case does not establish moving-target grasping capability.

Level	Source-defined setting
Easy	Original embedded difficulty table not retrieved.
Medium	Original embedded difficulty table not retrieved.
Hard	Original embedded difficulty table not retrieved.

Proposed diagnostics. Repeat identification, missed submissions and mistaken identity binding. These are proposed reporting fields, not measured results.

Boundary. Keep the stopped-tray rule. Continuous motion and recirculation belong to CP01.

A.2. P – Incremental meal orders

Suite. Cognitive Core; existing definition.

Instruction. Read each briefly displayed order specifying a container color, ingredient image and quantity. Every order adds a new requirement rather than replacing earlier orders. Remember both cumulative demand and what has already been placed in each container, and choose the execution order. New orders may arrive during manipulation. After ALL ORDERS SENT, finish the remaining ingredients and stop with exactly the requested contents in every container.

Task requirement. Maintain separate demand and delivery ledgers while orders and actions interleave. Opaque container walls naturally obscure previously deposited ingredients; the display does not maintain cumulative totals for the model.

Observation boundary. Transient order displays, visible container identities and the manipulation stream. The model must retain past order and placement evidence; ground-truth remaining quantities are not exposed.

Completion condition. After the end signal, every container has exactly the cumulative requested ingredient types and counts, with no omissions or extras.

Level	Source-defined setting
Easy	Original embedded difficulty table not retrieved.
Medium	Original embedded difficulty table not retrieved.
Hard	Original embedded difficulty table not retrieved.

Proposed diagnostics. Missed orders, duplicate deposits, container-binding errors and premature termination.

Boundary. Moving compartmented containers and deposit-confirmation mechanisms in CP02 are proposed additions.

A.3. S – Color-layout restoration after tray rotation

Suite. Cognitive Core; existing definition.

Instruction. Briefly observe red, yellow, blue and green cues on four positions of a rotating tray. Remember the color-position mapping. After the cues disappear, the four positions look alike; track the tray while it rotates. Once the tray stops and remains stationary, put each colored block from the front supply area into its corresponding position to restore the original layout.

Task requirement. Preserve a hidden color mapping while transforming between tray-relative positions and the visible world frame.

Observation boundary. The cue phase, observable tray motion and the final stationary tray. The original color mapping is not displayed during placement.

Completion condition. All four blocks occupy their original tray-relative color positions after release.

Level	Source-defined setting
Easy	Original embedded difficulty table not retrieved.
Medium	Original embedded difficulty table not retrieved.
Hard	Original embedded difficulty table not retrieved.

Proposed diagnostics. Color-position binding and tracking errors, separated from placement errors when the evidence permits.

Boundary. The original tray stops before placement. Continuous rotating placement is CP03, not an existing property of S.

A.4. F – Unknown button mapping and stage navigation

Suite. Cognitive Core; existing definition.

Instruction. Explore unlabeled buttons controlling the directions of a sliding stage. Press and release a button, observe the resulting motion and remember its direction. One effective press produces one move; holding a button does not repeat the move, and commands beyond the boundary produce no displacement. After exploration, the stage resets to a stated starting point and a green marker indicates the goal. Reach it by a shortest path using the learned mapping. Move the arm away when needed to see the stage and goal.

Task requirement. Infer a fixed within-episode action mapping from valid interventions, retain it across a reset and compose a route.

Observation boundary. Button appearance, press/release motion, stage displacement and the visible goal. Buttons carry no letters or direction hints. The mapping may change between instances, but not within an instance.

Completion condition. Reach the indicated goal by a shortest valid route after exploration, respecting press-release semantics. Exploration and navigation costs should be logged separately.

Level	Source-defined setting
Easy	One-dimensional track; two buttons; shortest path 1–3 steps.
Medium	3 by 3 grid; four directions; shortest path 2–3 steps.
Hard	4 by 4 grid; four directions; shortest path 4–6 steps.

Proposed diagnostics. Mapping errors, invalid boundary probes, redundant moves and failed physical presses.

Boundary. No within-episode mapping drift. CP04 adds step-size modes, control windows and explicit validation of probe effectiveness.

A.5. M – Workpiece tracking and route replay

Suite. Cognitive Core; existing definition.

Instruction. Observe a blue target workpiece on the rear carriers and remember its initial station. After covers close, the carriers, covers and their contents exchange positions. Track the target and remember every station it visits. Fixed left-to-right station positions correspond to submission pads A–D, or A–E on Hard. After all swaps stop, uncover the target, retrieve it and place it first on its initial station pad, then on each pad in its observed route. Release and stabilize it at every visit. A repeated station requires another distinct pick-and-place.

Task requirement. Track an occluded entity and retain an ordered route with repeated visits. Swaps not involving the target add no route node; temporary staging and handovers add no submission node.

Observation boundary. The initial identity, cover and carrier motions, fixed station labels and replay actions. Ground-truth target identity under a cover and the complete route remain evaluator-only.

Completion condition. The actual stabilized submission sequence exactly matches the observed target route, including its initial position and repeated visits.

Level	Source-defined setting
Easy	4 stations; 3 total swaps and 3 target moves; no irrelevant swap or revisit; 4 replay placements.
Medium	4 stations; 5 swaps, 4 target moves, 1 irrelevant swap and at least 1 revisit; 5 placements.
Hard	5 stations; 7 swaps, 5 target moves, 2 irrelevant swaps and at least 2 revisits; 6 placements.

Proposed diagnostics. Route edit distance, identity loss, omitted repeated visits and false placement completion.

Boundary. Station labels stay fixed in space. CP05 introduces moving station trays and timed handovers.

A.6. FR1 – Conveyor grasping

Suite. Motor Core; existing definition.

Instruction. Observe a cardboard box moving on a conveyor. Grasp it before it exits the reachable region and put it in the collection tray beside the conveyor. The belt continues to move during the grasp. Adapt the approach to the box motion, then release the gripper so the complete box remains stable in the collection tray.

Task requirement. Intercept a moving object under a finite physical opportunity window, then complete a stable transfer.

Observation boundary. RGB observations of the moving box, gripper and collection tray, plus robot state. Target motion and grasp-window truth are not exposed as oracle object state.

Completion condition. The box is caught before leaving the reachable region and is completely, stably deposited after release.

Level	Source-defined setting
Easy	Original embedded difficulty table not retrieved.
Medium	Original embedded difficulty table not retrieved.
Hard	Original embedded difficulty table not retrieved.

Proposed diagnostics. Window misses, grasp success, stable delivery and observation-to-action delay.

Boundary. The source calls this a motor-frequency case. A claim about real-time agent control additionally requires a non-pausing environment clock.

A.7. FR2 – Repeated responses to illuminated buttons

Suite. Motor Core; existing definition.

Instruction. Among three identical unlabeled buttons, press the one that is currently illuminated within the specified response deadline, then release it. Complete all 10 prompts without pressing an unlit button. Time runs from illumination to the effective press threshold. The light goes off after a valid press; release must occur within 1 second. The next prompt follows 0.3 seconds after confirmed release. The same button may light again, but continued holding cannot count as a new response.

Task requirement. Repeated visual event detection, timely contact and release, with explicit separation of independent responses.

Observation boundary. Visible lights, button/gripper motion and robot state. The agent receives no future prompt sequence.

Completion condition. All 10 prompts produce a valid press on the correct button within the configured deadline and a timely release; wrong-button presses are violations.

Level	Source-defined setting
Easy	Response-deadline table not retrieved; shared 10-prompt and release rules remain known.
Medium	Response-deadline table not retrieved.
Hard	Response-deadline table not retrieved.

Proposed diagnostics. Response latency, deadline misses, wrong presses, release violations and repeated-prompt errors.

Boundary. The 1-second release limit and 0.3-second inter-prompt delay are source-defined. Missing response deadlines must not be inferred from them.

A.8. PR1 – Linear resistance adjustment

Suite. Motor Core; existing definition.

Instruction. Move the insulated handle of a sliding rheostat to the resistance specified by the instruction. Read the orange indicator line against a 0–40 ohm scale. Repeated left/right corrections are allowed. Release the handle and withdraw, then keep the reading within tolerance for 1 second. The target is given in language; no green target region is drawn on the device.

Task requirement. Linear contact adjustment, accurate stopping and stability after release. There is no object to measure and no electrical experimentation requirement.

Observation boundary. The visible scale and indicator, arm motion and robot state. No oracle scalar error-to-target is returned to the policy.

Completion condition. After release and withdrawal, resistance remains in the specified interval continuously for 1 second.

Level	Source-defined setting
Easy	One target; resistance tolerance ± 1 ohm, corresponding to ± 10 mm.
Medium	One target; ± 0.5 ohm, corresponding to ± 5 mm.
Hard	One target; ± 0.2 ohm, corresponding to ± 2 mm.

Proposed diagnostics. Post-release residual, correction count, overshoot and stability violations.

Boundary. These are stated design tolerances, not measured robot accuracy.

A.9. PR2 – Sequential voltage-dial setting

Suite. Motor Core; existing definition.

Instruction. Turn the pointer on a tabletop voltage dial to each instructed target. Regrasping, release and reverse corrections are allowed. Each target must remain stable after release before advancing to the next; the dial is not reset between targets. The scale spans 0–260 V, with numbers every 20 V and minor marks every 5 V. Voltage is a predefined linear function of dial angle, not an additional circuit task.

Task requirement. Angular positioning and repeated direction reversal while retaining the actual state between targets.

Observation boundary. The dial scale and pointer, the current instruction and robot state. Target values are not marked on the scale.

Completion condition. Every target is reached and stably maintained after release in the required sequence. Angular/voltage tolerances and hold duration still require calibration.

Level	Source-defined setting
Easy	One target; wider tolerance, to be calibrated; one adjustment.
Medium	Two targets; narrower tolerance, to be calibrated; second target reverses direction.
Hard	Three targets; same tolerance as Medium; adjacent targets alternate increasing and decreasing.

Proposed diagnostics. Angular/voltage residual, reverse-correction behavior, regrasp count and stability.

Boundary. Dial size, appearance, friction and damping stay fixed. The source explicitly excludes backlash and reverse dead travel.

A.10. PR3 – Sample-tube alignment and insertion

Suite. Motor Core; existing definition.

Instruction. Pick up one sample tube lying in the supply tray, bring it upright and align its tip with the target hole marked by a green ring on the rack. Insert slowly along the hole axis to the required depth, correcting position and orientation if necessary. Release and withdraw so the tube remains stable in the target hole.

Task requirement. Functional alignment at the opening and sustained axial alignment over a constrained insertion travel.

Observation boundary. RGB observations of the tube, marked hole and manipulation, plus robot state. Hole/tube ground-truth poses and completion depth are reserved for evaluation.

Completion condition. The specified tube reaches the required depth and remains stable in the marked hole after release and withdrawal.

Level	Source-defined setting
Easy	One tube and one hole; wider clearance and shorter effective guide length.
Medium	Narrower clearance; guide length unchanged from Easy.
Hard	Clearance unchanged from Medium; longer effective guide length.

Proposed diagnostics. Alignment residual, insertion depth, jamming, retries and post-release stability.

Boundary. Grasp conditions and initial-pose ranges stay fixed. Exact clearance and guide-length values are not provided in the current text.

B. Coupling Task Cards

All twelve cases are design proposals. Episodes below are constructed examples, not measured rollouts. Intent/effect fields describe task dependencies; they are not mandatory exposed modules for an end-to-end policy. Main-track feedback must be observable through the declared RGB/proprioception channels; additional instrumented channels are separate conditions.

B.1. CP01 – History-conditioned dynamic sorting

Question. Can a history-selected target be handled within a physical grasp window, with the actual outcome carried into subsequent task state?

Instruction. Observe an initial batch of objects. In each subsequent batch moving continuously through the work area, grasp the unique item seen before and place it in that batch’s submission tray. A missed target may return once on a recirculation belt. A batch is complete only after the correct object is stably deposited.

Scene. A single arm, conveyor, batch-specific trays and a one-return recirculation path. Batch boundaries are visible; a returning object does not create a new order.

Model-visible evidence. Batch cues, visual object appearance and position, gripper state and submission-tray images. Empty grasps, slips and successful deposits have observable consequences.

Evaluator-only state. Object identity truth, repeated-item labels, exact grasp windows and verified submission state remain evaluator-only.

Decision and execution. Maintain both a seen-identity set and a pending-batch ledger. Choose the repeated item, retain unresolved submissions after a miss and decide whether to wait for its return. Track, approach, grasp and transfer the moving object, then release it stably within the permitted window.

Two-way dependency. Intent to action: Target identity/visible features, batch and tray binding, current opportunity window, completion condition and return budget. Effect to continuation: Observed held object, evidence of deposit, remaining opportunity and return status, with the associated observation time.

Example event		Required continuation
Batch 1 contains A and B.		Observe only; remember both identities.
Batch 2 contains B and C; the grasp of B misses.		Keep batch 2 pending because the gripper and tray remain empty.
B returns as batch 3 begins.		Deposit B into batch 2’s tray, verify it, then close that pending item.
Level	Proposed controlled change	
Easy	Short history and two submission batches; a wide fixed window; one recognizable miss requires waiting for the return.	
Medium	More batches and similar-looking objects; retain Easy’s window; maintain seen and undelivered records separately.	
Hard	Retain Medium’s history load; shorten the grasp window only; resolve interleaved return and new-batch events.	

Success. Every required repeat item is deposited in the correct batch tray within the declared return budget. A closed gripper or finished command is not a completed submission.

Measurements. Batch completion, identity/batch errors, missed windows, premature completion and extra return/recovery cost.

Failure distinction. The correct target B is selected but a stale approach misses it: execution failure. Empty-gripper and empty-tray evidence is observed but batch 2 is marked complete: feedback-use failure.

Diagnostic intervention. Delay a delivery-confirmation channel while retaining the underlying observation stream; measure duplicate or skipped submissions. Any added feedback channel is declared separately from RGB-only evaluation.

Weak-coupling counterexample. An offline repeat-item answer followed by a controller that manages all batches and returns internally removes the progress-update question.

Reuse and additions. Reuses E’s cross-batch identity rule and FR1’s moving grasp. Recirculation, batch-specific completion and event-linked ledgers are additions; E itself uses stopped trays.

Component sources. E; FR1. Sources support components, not the complete coupled task.

B.2. CP02 – Incremental orders with moving-bin portioning

Question. Can cumulative demand and actual deliveries remain distinct while precise deposits occur in moving-container windows?

Instruction. Read briefly displayed incremental orders. Containers pass the work station on a loop. Remember each container’s cumulative requirements and deposit the correct ingredient units into its assigned small compartment. Use observable misses, wrong deposits and drops to revise the remaining work.

Scene. Compartmented containers, discrete ingredient blocks, a circulating conveyor and a visible inspection zone. Units are counted blocks, not an unvalidated liquid-volume estimator.

Model-visible evidence. New orders, visible container identity, deposit images and observable evidence at the inspection zone. Walls obscure accumulated contents and the display does not maintain the ledger.

Evaluator-only state. Cumulative demand truth, compartment counts and error attribution remain with the evaluator. A sensor cue may describe the current effect but cannot reveal the remaining answer.

Decision and execution. Maintain demand and delivered counts; allocate current windows to container/ingredient pairs; revise the execution order or recovery actions from real deposits. Requested quantities remain unchanged. Follow the target compartment, release within tolerance and distinguish a stable deposit from a slip, retained grasp or neighboring-compartment miss.

Two-way dependency. Intent to action: Container, compartment, ingredient, quantity for this action, release tolerance, window and operation identifier. Effect to continuation: Observed entry or drop, uncertain outcomes, whether the object remains held and when the container leaves view.

Example event		Required continuation
A requests two red blocks, then one blue block. The first red block slips into a recovery region.		Retain cumulative demand of two red and one blue. Do not decrement A’s remaining red demand merely because release finished.
B’s window arrives before A returns.		Serve B’s valid demand, update only B, then resume A.
Level	Proposed controlled change	
Easy	One container and short orders; large compartments and a wide window; distinguish release from entry.	
Medium	Two containers with new orders during execution; hold speed and tolerance fixed; interleave demand/delivery updates.	
Hard	Keep Medium’s orders; narrow the compartment only, with the window unchanged; recover one drop without double-serving another container.	

Success. After the end signal, all valid orders are satisfied exactly, with no extra ingredients or wrong compartments. Unrecoverable wrong deposits fail; legal retrieval and refill rules are fixed before evaluation.

Measurements. Order completion, demand/delivery mismatch, duplicate deposits, wrong compartments, within-window effective deposits and refill cost.

Failure distinction. A correct red-compartment target is missed by the release: execution failure. The missed deposit is nevertheless subtracted from demand: feedback-use failure.

Diagnostic intervention. Remove or delay a declared deposit-confirmation channel while retaining orders, cameras and inspection costs; inspect cross-container event binding.

Weak-coupling counterexample. Expanding every order into a fixed action list and counting contents only at the end hides intermediate ledger updates.

Reuse and additions. Reuses P’s additive orders, FR1’s continuous motion and PR3-inspired placement tolerance. Moving compartments, deposit events and refill rules are proposed additions.

Component sources. P; FR1; PR3. Sources support components, not the complete coupled task.

B.3. CP03 – Remembered-layout restoration on a moving tray

Question. Can remembered tray-relative targets be transformed into current coordinates and revised after actual placement?

Instruction. Observe a brief color layout on a tray. After the cues disappear, the tray continues rotating. Put colored blocks into their remembered positions, verify each actual placement and, within budget, retrieve misplaced blocks to restore the layout.

Scene. A four-position turntable with trackable structural features, colored blocks and a recovery buffer. Removing color answers must not remove all pose cues and make the task unobservable.

Model-visible evidence. The initial color mapping, continuing tray-pose cues, block motion and settled placement.

Evaluator-only state. The original mapping, exact rotation and ground-truth occupancy are evaluator-only; current positions do not reveal the hidden colors.

Decision and execution. Preserve the color-to-tray mapping, select a block and update unfinished or incorrectly occupied positions from observed effects; choose retrieval or reorder the remaining placements. Track the moving position, compensate release delay and achieve stable placement in a small region without stopping the tray.

Two-way dependency. Intent to action: Color, tray-relative slot and reference frame, release window and stable-placement requirement. Effect to continuation: Observed occupied slot, stability/slip evidence, blocked target positions, placement identity and feedback time.

Example event	Required continuation
Red belongs to slot 1; blue belongs to slot 3. A late release puts red in slot 2.	Keep red bound to tray slot 1 as its world coordinates change. Keep red unfinished and retrieve it instead of marking slot 1 complete.
Blue now has the better window.	Place blue first, then red, without changing the remembered target layout.

Level	Proposed controlled change
Easy	A small set of remembered bindings; slow constant rotation and wide tolerance; verify before choosing the next block.
Medium	Add a confusable binding while holding motion/tolerance fixed; recognize and clear one wrong occupancy.
Hard	Retain Medium's layout and tolerance; introduce observable speed variation; refresh target coordinates and placement order.

Success. All blocks stably occupy their original tray-relative positions within the repair budget. The evaluator scores in the tray frame and records whether rotation remained continuous.

Measurements. Layout completion, slot-binding errors, release/landing residual, uncorrected occupancy errors and local repair cost.

Failure distinction. The correct slot is chosen but release lag lands red in slot 2. A second failure occurs if the system then treats slot 1 as complete and ignores the observed occupancy conflict.

Diagnostic intervention. Hold a coordinate transform stale for one action, or delay placement evidence, to examine stale-target and occupancy propagation.

Weak-coupling counterexample. Answering a spatial-memory question and waiting for the tray to stop removes the continuous transformation and timing requirement.

Reuse and additions. Reuses S's hidden layout and moving-placement primitives. Continuous motion, misplacement retrieval and occupancy-dependent replanning are new.

Component sources. S; [PlaceSphereEnv](#). Sources support components, not the complete coupled task.

B.4. CP04 – Timed calibration with unknown controls

Question. Can the system distinguish a wrong control hypothesis from a physically ineffective press?

Instruction. Unlabeled buttons control forward motion, reverse motion or a step-size mode. Use a limited number of probes to infer the rules, then move the pointer into a target region during the allowed input window. Release after every press. Recheck uncertain effects instead of interpreting an ineffective contact as evidence about the rule.

Scene. A fixed panel, graduated stage, visible input-window indicator and observable trigger indicator. The window limits input time; the panel itself does not move.

Model-visible evidence. Pointer displacement, actual press/release, a visible trigger cue and time-window information. Button direction and step-size mapping are not displayed.

Evaluator-only state. The exact mapping, contact threshold and fault class remain evaluator-only. Rules stay fixed within an episode.

Decision and execution. Maintain mapping hypotheses, assess probe validity, choose coarse/fine adjustments and revise from residuals. Reach the chosen button, cross its effective threshold and release within the permitted window.

Two-way dependency. Intent to action: Control to probe, allowed press pattern, opportunity window and the effect that would distinguish hypotheses. Effect to continuation: Visible trigger evidence, displacement/residual, release state and feedback time.

Example event	Required continuation
Button A's direction is unknown; the first contact is too shallow.	Keep the mapping unresolved because neither trigger nor displacement was observed.
A valid retry moves the stage right.	Accept this as evidence for A's direction.
A later adjustment overshoots.	Use the inferred fine mode and the actual residual to approach the target.

Level	Proposed controlled change
Easy	Two unknown directions; wide window and target region; require valid evidence before updating the mapping.
Medium	Add one step-size mode; retain the physical window/tolerance; choose coarse or fine control from residuals.
Hard	Retain Medium's rule complexity and window; tighten target tolerance; recover from an ineffective press without corrupting the mapping.

Success. The stage enters and remains in the target region within budget. Probes and control actions are separately charged; failed presses do not create unlimited free retries.

Measurements. Valid-probe use, execution-to-rule misattribution, position residual, missed windows and propagated mapping errors.

Failure distinction. The direction is correctly inferred but contact never crosses the threshold. Treating the observed lack of a valid trigger as proof that A is useless corrupts the task model.

Diagnostic intervention. Hide the declared trigger indicator while retaining motion images, or delay the motion observation; measure uncertainty and event mismatch.

Weak-coupling counterexample. Making every probe automatically successful and handing a known mapping to the controller removes the distinction between inference and contact failure.

Reuse and additions. Reuses F, PR1 and FR2. Step-size modes, input windows and trigger cues are added; the original F mapping still does not drift.

Component sources. F; [sawyer_door_lock_v3](#); [sawyer_door_unlock_v3](#); PR1; FR2. Sources support components, not the complete coupled task.

B.5. CP05 – Remembered routes with timed station handovers

Question. Can a route and its execution index stay synchronized with actual handovers, including revisits and failed attempts?

Instruction. Observe the target workpiece’s station route. Replay it by placing the workpiece onto moving station trays when their windows open, including repeated visits. Each node requires stable release and observable handover evidence. After a failed handover, wait for the same station’s next legal opportunity.

Scene. Fixed station identities, periodically arriving trays, a target workpiece and a temporary buffer. Station identity is not the instantaneous tray position.

Model-visible evidence. The demonstrated route, station labels, window cues, handover images and whether the workpiece remains held.

Evaluator-only state. Route truth, verified prefix length and handover-success labels belong to the evaluator.

Decision and execution. Maintain the complete route and its confirmed prefix; select the next station, wait or retrieve. Repeated station visits remain separate route nodes. Track the tray, place and release stably within the window, and recover a workpiece dropped in the buffer.

Two-way dependency. Intent to action: Station identity, route-node index, tray-relative placement target, window and release condition. Effect to continuation: Held-object state, actual placement/stability, associated node and observation time, and the next visible opportunity.

Example event	Required continuation
For route A-B-A-C, the first A is complete; transfer to B fails. An A tray becomes available first. B is stably completed.	Keep B as the active route node. Retrieve the workpiece and wait for B; do not skip to the repeated A. Advance to the second A, then C.

Level	Proposed controlled change
Easy	Three-node route; large trays and wide windows; advance only after release.
Medium	Add a revisit and an irrelevant observed swap; keep physical conditions fixed; preserve the route index after failure.
Hard	Retain Medium’s route; shorten the handover window only; disambiguate delayed evidence using its node and time.

Success. The actual stable handover sequence equals the demonstrated route, including repeats. Buffer placements do not count as route nodes; failed nodes cannot be skipped.

Measurements. Route edit distance, handover success, wrong advancement/rollback, stale-feedback binding and recovery waiting cost.

Failure distinction. B is the correct next station, but a late release misses its tray. Advancing to the repeated A despite the failed B handover produces an incorrect route.

Diagnostic intervention. Delay a confirmation or remove a declared node-association cue while retaining other evidence; inspect repeated-station ambiguity.

Weak-coupling counterexample. Passing the complete route to an executor that hides every retry prevents assessment of route-state synchronization.

Reuse and additions. Reuses M’s fixed labels and revisit semantics, plus handover and placement primitives. Moving windows, buffer recovery and node-linked logs require new implementation.

Component sources. [M](#); [TwoArmHandover](#); [S](#); [PlaceSphereEnv](#). Sources support components, not the complete coupled task.

B.6. CP06 – Recipe-driven multistage precision adjustment

Question. Can a remembered recipe govern precise adjustment, with measured stability rather than command completion enabling the next stage?

Instruction. Read a briefly displayed equipment recipe and perform its slider and dial settings in order. Release and verify stability at each stage. If a reading is invalid, correct only the parameters still out of specification while preserving conditions already verified.

Scene. A PR1-like slider, PR2-like dial, stage indicators and a confirmation control. Dependencies are implemented as an explicit task state machine, not assumed electrical coupling.

Model-visible evidence. The transient recipe, current scales, stage lamps and declared acceptance observations. The full recipe does not remain visible.

Evaluator-only state. The complete recipe, exact stability truth and authoritative stage state remain evaluator-only.

Decision and execution. Remember targets and dependencies, choose the active parameter and decide whether to correct, reobserve or advance. Stop at the target position/angle, reverse when required and verify readings after release, including any physical drift.

Two-way dependency. Intent to action: Parameter, target, tolerance, required hold, stage identifier and constraints on disturbing earlier settings. Effect to continuation: Observed post-release reading, stability evidence and any disturbed prerequisite.

Example event		Required continuation
The slider must pass before the dial is set; release causes slider drift.		Remain at stage 1 and readjust.
The slider passes, but later dial contact disturbs it.		Invalidate the affected prerequisite and repair it.
Both conditions are stable.		Confirm stage 2 without repeating unrelated valid adjustments.

Level	Proposed controlled change
Easy	Two simple stages; wide tolerance and fixed hold; advance from post-release evidence.
Medium	Three stages with a reversal or one recipe revision; keep tolerance fixed; preserve still-valid settings.
Hard	Keep Medium’s recipe; tighten one parameter tolerance; restore a prerequisite disturbed by later contact.

Success. All stages follow their valid dependencies. At each stage submission, all parameters required by that stage must simultaneously satisfy their stability conditions. Reaching only the last target value is insufficient.

Measurements. Stage completion, premature advancement, parameter residual, unnecessary readjustment and missed prerequisite invalidation.

Failure distinction. The intended value is correct but release produces drift. Advancing despite the observed out-of-tolerance value turns execution error into a dependency-state error.

Diagnostic intervention. Compare effect-confirmed progression with command-completion progression; delay a declared prerequisite-change cue.

Weak-coupling counterexample. A controller receiving the complete recipe and internally managing every stage hides the progress-decision problem.

Reuse and additions. Reuses PR1, PR2 and P-like state maintenance. Recipe dependencies and invalidation checks are new; PR2 is not retrospectively assigned backlash.

Component sources. P; [open_drawer](#); [place_in_drawer](#); [PR1](#); [PR2](#). Sources support components, not the complete coupled task.

B.7. CP07 – Resource-constrained moving-part assembly

Question. Can resource and process plans be revised from the real outcomes of moving-part acquisition and insertion?

Instruction. Use two bills of materials to collect and insert parts for different kits. Parts arrive in batches on a conveyor, inventory is shared and replenishment is limited. After a failed grasp or insertion, reassess available parts, unfinished kits and remaining opportunities.

Scene. Moving supply, two kit fixtures, shared parts, a defect-isolation area and replenishment events. A failed insertion does not imply a damaged part without inspection evidence.

Model-visible evidence. Bills of materials, explicit priorities, arrival cues, part appearance, manipulation and inspection observations.

Evaluator-only state. Inventory/connection truth, undetected defects and future arrivals remain evaluator-only; there is no implicit unlimited supply.

Decision and execution. Decompose each kit, allocate shared stock, select available targets and update free, held, in-process or rejected resources from actual events. Grasp the correct moving part and align, insert and release it within the required tolerance.

Two-way dependency. Intent to action: Part-kit binding, active operation and prerequisites, grasp window and insertion-completion condition. Effect to continuation: Acquisition evidence, connection state, evidence supporting retry or rejection and actual consumption/return.

Example event	Required continuation
A has priority; only one valid shared pin is present. Insertion jams; inspection finds the pin intact.	Allocate the pin to A and mark it in process after acquisition. Retain its allocation and retry alignment; do not create free stock for B.
A passes and replenishment arrives.	Close A and allocate the newly available part to B.

Level	Proposed controlled change
Easy	One complete kit; slow supply and wide insertion tolerance; separate acquisition from connection.
Medium	Two kits share a prioritized resource; hold physical difficulty fixed; update inventory after returns/failures.
Hard	Retain Medium's bill of materials; narrow insertion clearance only; handle interleaved replenishment and delayed inspection.

Success. Kits satisfy their bills of materials and connection requirements, resource accounting is conserved and explicit priorities/budgets are respected.

Measurements. Kit completion, inventory mismatch, invalid process advancement, grasp/insertion success and feedback-driven schedule repair.

Failure distinction. A correctly allocated pin is missed or jams. Counting a missed pin as acquired, or discarding an observed intact return as damaged, corrupts subsequent allocation.

Diagnostic intervention. Replace actual consumption updates with intended consumption in a declared ablation; suppress a declared return-inspection channel.

Weak-coupling counterexample. Prepositioning unlimited material and making insertion always succeed removes the physical-to-resource feedback.

Reuse and additions. Uses kit-assembly primitives, FR1 and PR3. Cross-operation resource accounting, inspection and finite replenishment are new mechanisms.

Component sources. [AssemblingKitsEnv](#); [FR1](#); [PR3](#). Sources support components, not the complete coupled task.

B.8. CP08 – Inspection-driven assembly and local repair

Question. Can an observed connection defect trigger the necessary local repair and revalidation without undoing unrelated valid work?

Instruction. Assemble several connections with dependencies. Inspection may reveal partial insertion, misalignment or an unsecured connection. Locate the defect, repair the necessary connections and recheck the affected structure while preserving independent valid subassemblies.

Scene. A repairable structure, role-specific connectors, a visible inspection station and buffer. The initial design uses geometric/locking proxies, not unverified electrical or torque measurements.

Model-visible evidence. Part roles, assembly images, observable inspection indications and depth/contact evidence. Inspection describes the condition, not a complete repair program.

Evaluator-only state. Connection truth, hidden defect labels and a minimal repair subgraph are evaluator-only. Multiple legitimate repair paths are allowed.

Decision and execution. Maintain dependencies, interpret defects, select withdrawal, alignment, replacement or rollback, and identify checks invalidated by the repair. Withdraw, align and reconnect without unnecessarily disturbing neighboring connections, then obtain fresh inspection evidence.

Two-way dependency. Intent to action: Connection to repair, allowed removal scope, dependency constraints, target depth/orientation and completion condition. Effect to continuation: Actual removal/reconnection extent, observed residual, neighboring disturbance and renewed inspection evidence.

Example event		Required continuation
A supports B; C is independent. Inspection finds B partly inserted. Repairing B slightly disturbs A. A and B pass reinspection.		Mark B for repair while retaining C as valid. Schedule checks of A and B rather than reusing A's old pass. Resume remaining assembly and record local repair cost.
Level	Proposed controlled change	
Easy	Two connections with clear defect localization; wide tolerance; advance only after inspection.	
Medium	Add an independent component and a shared support; retain physical difficulty; choose the necessary repair scope.	
Hard	Retain Medium's structure; tighten one connection tolerance; account for a neighboring connection disturbed by repair.	

Success. All structural conditions hold and every check invalidated by repair passes again. Score legitimate repair paths by outcome and cost, not one prescribed sequence.

Measurements. Assembly completion, defect propagation, unnecessary disassembly, missed rechecks and local repair actions/time.

Failure distinction. A correctly selected connector is only partly inserted. After repair disturbs A, updating B alone and retaining a stale pass for A is a feedback-use failure.

Diagnostic intervention. Mask a declared inspection-message channel while retaining the inspection action/cost; remove affected-neighbor hints while preserving RGB.

Weak-coupling counterexample. Resetting the entire structure and replaying the same script after every defect is a useful baseline but does not demonstrate local dependency recovery.

Reuse and additions. Draws on FurnitureBench-style assembly and insertion primitives. Repairable geometry, inspection state and dependency invalidation must be implemented; geometric alignment is not electrical continuity.

Component sources. [Chair](#); [PR3](#); [plug_in_charger](#); [PlugChargerEnv](#). Sources support components, not the complete coupled task.

B.9. CP09 – Concurrent preparation and quantitative delivery

Question. Can preparation stages, quantities and order identities stay consistent after actual losses and asynchronous completion?

Instruction. Prepare and deliver specified amounts for two containers. Wait for evidence that preparation is complete. Pouring and transport may change the remaining amount. Deliver the correct container and quantity within budget; starting a device or finishing a pour does not establish completed delivery.

Scene. Two orders, visible equipment states, source vessels, receiving cups and a delivery region. Quantities need an independently measurable liquid or particle realization; stage durations are defined by the task state machine.

Model-visible evidence. Orders, equipment indicators, observable level or scale readings, spills and container identity. Future completion times and perturbation outcomes are not revealed.

Evaluator-only state. Recipe truth, complete device state and particle-level quantity remain evaluator-only. A visible quantity reading does not directly state the missing amount.

Decision and execution. Maintain order stages and container bindings, use waiting intervals for other work and choose refill, remake or delivery from measured effects. Control pour amount and stopping, transport stably, correct tilt/slip and verify the amount after settling.

Two-way dependency. Intent to action: Order/container binding, target quantity and tolerance, preparation prerequisites, delivery location and deadline. Effect to continuation: Observed completion signal, settled quantity, transport loss and identity/time evidence.

Example event		Required continuation
A is preparing while B can be staged.		Prepare B's cup without treating A's start as completion.
A reaches its pour target but spills in transport.		Return A to pending refill after observing the lower settled amount.
B finishes preparing meanwhile.		Set A down, deliver B, then refill A; close each order separately.
Level	Proposed controlled change	
Easy	One order, explicit completion cue, wide quantity tolerance and short transport.	
Medium	Two orders with different stage durations; unchanged physical difficulty; bind asynchronous events correctly.	
Hard	Retain Medium's orders; tighten quantity tolerance only; revise delivery after an observed loss.	

Success. Correct containers are delivered with valid preparation conditions and final quantities, using real refill resources within budget. Particle proxies and physical fluid performance are reported separately.

Measurements. Order completion, falsely completed stages, quantity residual, wrong-order updates, loss-recovery cost and delivery timeouts.

Failure distinction. The correct target amount is selected but residual flow or transport tilt violates tolerance. Reusing an earlier valid-pour record despite a later loss incorrectly closes the order.

Diagnostic intervention. Mask a declared post-transport reading while preserving inspection cost, or delay a completion message and examine order-state contamination.

Weak-coupling counterexample. Checking only whether a cup contains something, or replacing completion evidence with a fixed wait, hides stage and quantity errors.

Reuse and additions. Combines coffee/heating process primitives, quantitative pouring and stable transport. Asynchronous order binding and final-quantity rechecks are new. Waiting alone does not establish frequency capability; timed pour-stop or disturbance correction must be present.

Component sources. [PrepareCoffee](#); [HeatMultipleWater](#); [pour_liquid_into_cup](#); [pour_water](#); [TwoArmLift](#). Sources support components, not the complete coupled task.

B.10. CP10 – Precision packing with shared-space constraints

Question. Can a packing plan be revised from actual placement and access-space changes instead of executing a fixed order?

Instruction. Pack several objects into a box under functional orientation and position constraints with limited temporary storage. After every placement, inspect the actual pose and remaining access routes. If the next object cannot enter as planned, move only the necessary objects while preserving valid work.

Scene. A box with narrow access, differently shaped functional objects and one limited buffer. Multiple legal final arrangements are allowed.

Model-visible evidence. Target relations, box geometry, placed-object appearance, approach and contact observations; the agent may reobserve occluded regions.

Evaluator-only state. Exact occupancy, collision truth and the set of feasible layouts remain evaluator-only. No optimal arrangement or remaining action sequence is given.

Decision and execution. Choose arrangement and order, maintain actual occupancy/buffer use and select local reordering after a physical deviation. Place, retrieve and reorient precisely in confined space without unacceptable contact or unnecessary disturbance.

Two-way dependency. Intent to action: Object, required orientation, permitted region, access/contact constraints and earlier results to preserve. Effect to continuation: Observed pose, blockage evidence, displaced neighbors and buffer occupancy.

Example event		Required continuation
A should leave room for B, but lands slightly off target.		Recognize that B’s approach is narrowed instead of forcing the original insertion.
The temporary buffer is free.		Retrieve A, then place B.
A can now enter from above.		Reinsert A in a legal arrangement and record the extra moves.
Level	Proposed controlled change	
Easy	Two objects without buffer conflict; wide placement regions; update actual occupancy.	
Medium	Three objects share one buffer; unchanged tolerance; a deviation requires local reorder.	
Hard	Keep object/constraint counts fixed; narrow one access route; respect buffer capacity and valid earlier constraints during repair.	

Success. All objects meet final orientation/position constraints, no unfinished item remains in the buffer and no prohibited collision occurs. Legal alternative layouts are accepted.

Measurements. Packing completion, planned/actual pose difference, ineffective retries after blockage, extra relocations and buffer conflicts.

Failure distinction. A feasible planned layout becomes blocked by inaccurate placement. Repeatedly issuing the same B insertion after visible blockage is a failure to revise the plan.

Diagnostic intervention. Replace observed occupancy with intended pose in a declared ablation, or delay evidence of displaced neighbors.

Weak-coupling counterexample. Oversized free space or an executor that solves the complete packing problem eliminates the need for occupancy-dependent planning.

Reuse and additions. Uses packing, shared-workspace and precision-placement primitives. Narrow-access geometry, multiple-solution verification and occupancy feedback require new design.

Component sources. [pack_objects_into_box](#); [organize_table](#); [deposit_coin](#). Sources support components, not the complete coupled task.

B.11. CP11 – Interlocked cleaning with contact maintenance

Question. Can a cleaning plan respond to contact and equipment-state changes while preserving both process and final-state constraints?

Instruction. Turn off the equipment, prepare an appropriate tool and clean the required regions. Maintain allowed contact and stay within the permitted area. If the equipment changes state, the tool becomes ineffective or a region remains dirty, stop as needed, restore prerequisites and resume the unfinished work before final inspection.

Scene. Equipment with visible power state, cleaning regions, a tool area and a contact sensor or declared proxy. State changes are controlled simulation events. Physical force is reported only when the implementation and sensor support it.

Model-visible evidence. Equipment indicators, visible cleaning/tool condition and only the contact/force information allowed by the declared sensor configuration. No repair-action label is supplied.

Evaluator-only state. Interlock violations, exact contamination coverage and contact truth remain evaluator-only unless a matching public sensor legitimately measures them.

Decision and execution. Organize prerequisites and region order, revise unfinished areas from effects and decide to pause, change tool or revalidate. Maintain contact while traversing the surface and stop contact motion on a violated interlock; resume locally when permitted.

Two-way dependency. Intent to action: Active region, contact limits, required interlock, contact target/proxy and abort condition. Effect to continuation: Observed contact loss/overload, processed-area evidence, equipment-state time and changed tool effectiveness.

Example event	Required continuation
The device is off; cleaning A begins.	Accumulate only the visibly treated portion.
A controlled event turns the device on.	Withdraw promptly; keep A partial rather than completed.
Power-off is restored and verified.	Resume A's remaining area, then B, and verify process and final conditions.

Level	Proposed controlled change
Easy	One region, explicit interlock, slow planar contact and a fixed proxy.
Medium	Two regions plus one tool-preparation prerequisite; keep contact difficulty fixed; redo only the insufficiently cleaned region.
Hard	Keep Medium's plan; add one equipment-state change with geometry fixed; stop, restore prerequisites and resume.

Success. All required regions are clean and declared interlock/contact conditions hold. Never initiate wiping while the device is on. For an unexpected off-to-on transition during contact, stop and withdraw within a response window fixed before evaluation; restart only after power-off is verified. A correct stop without eventual completion is not full success.

Measurements. Cleaning completion, interlock violations, state-change-to-stop delay, contact loss, overstated coverage and post-recovery completion.

Failure distinction. The right region/tool is selected but surface variation breaks contact. Marking the region complete after an abort and partial-coverage observation is a progress-update failure.

Diagnostic intervention. Mask declared coverage feedback while retaining RGB; delay high-level state information while keeping any declared low-level safety stop fixed and logged.

Weak-coupling counterexample. A complete cleaning script that internally handles all interlocks, regions and tool changes hides the agent's state-maintenance role.

Reuse and additions. IS-Bench supplies an equipment-off prerequisite and Wipe supplies contact-cleaning primitives. Mid-task state changes, regional progress and recovery are new. Force or additional sensor tracks must be separated from the RGB-plus-proprioception main track. This draft clarifies the transition rule with a pending, predeclared stop-response window; it is not a measured deadline.

Component sources. [clean_a_toaster_oven_toggled_on](#); [Wipe](#). Sources support components, not the complete coupled task.

B.12. CP12 – Demonstrated rhythmic tapping with resynchronization

Question. Can a remembered note sequence and current measure stay aligned with real contact events and a running beat?

Instruction. Watch one demonstration of key order and rhythm, then reproduce it with a tapping stick after the target demonstration is removed. Lift before each independent tap. After a missed or double tap, retry from the stated measure boundary within budget, preserving previously accepted measures. Complete every required measure.

Scene. A xylophone or key board, stick, visible public beat and measure-boundary cues. Only the target demonstration is removed; current timing remains observable and retry rules are explicit.

Model-visible evidence. Demonstrated targets/rhythm, the public beat, visually observable contact and stick lift. Audio, if used, is an explicitly separate sensor condition, not implicit extra input.

Evaluator-only state. The target note-event sequence, exact contact times and verified passed measures remain evaluator-only. The public beat does not reveal the next target key.

Decision and execution. Retain the demonstration and measure/note index; continue or restart the current measure from observed contacts while preserving a valid prefix. Hit the right key at the right time, lift between hits and produce observable evidence of a valid tap, miss or double hit.

Two-way dependency. Intent to action: Measure/note identifier, target key, contact window and lift/retry boundary. Effect to continuation: Observed key and contact time, independent contact count, lift state and evidence affecting the current measure.

Example event		Required continuation
The target measure is red-blue-blue; red and the first blue are tapped. The stick stays pressed on blue. The next public measure boundary arrives.		The next target remains blue but requires a lift and a new contact. Treat the second blue as missed; do not advance the measure index. Retry this measure only and advance after three valid independent taps.
Level	Proposed controlled change	
Easy	One short measure with few keys; slow beat and wide windows; advance from actual contacts.	
Medium	Two measures including a repeated key; retain tempo/window; preserve valid earlier measures during a retry.	
Hard	Keep Medium’s target sequence and key spacing; narrow timing windows only; resynchronize after a miss without skipping a note.	

Success. Within the declared retry budget, each target measure has at least one complete execution satisfying both order and timing. Failed attempts stay in the log and cost accounting.

Measurements. Measure/full-sequence completion, wrong keys, timing residual, repeated-key omissions, index drift and resynchronization cost.

Failure distinction. The correct key/window is intended but the tip hits a neighbor or does not lift. Advancing the index without a new contact shifts all later actions.

Diagnostic intervention. Compare command-count advancement with contact-confirmed advancement; delay a declared contact cue and inspect stale-event effects on restarted measures.

Weak-coupling counterexample. Leaving the target sequence visible, or scoring only whether every key was touched, removes memory, multiplicity and timing requirements.

Reuse and additions. Combines demonstrated-sequence and xylophone primitives. The public beat, measure retries and independent-contact scoring are proposed additions.

Component sources. [imitate_sorting_sequence](#); [play_Xylophone](#). Sources support components, not the complete coupled task.

C. Reserved Detailed Result Tables

The symbol — means *not evaluated*, not zero. These tables reserve the same twenty-one configurations as the main comparison. They contain no external benchmark scores. Difficulty, sensor permissions, adaptation setting and clock condition must be specified before filling any table. Unavailable configurations remain missing; genuinely inapplicable metrics will be marked N/A with an explanation.

Table 6: Cognitive Core success rates. All measurement cells are *not evaluated*.

Model / system	E	P	S	F	M	Mean
Coding agents + proposed Inspect EEF adapter						
GPT-6 Astra	—	—	—	—	—	—
Claude Opus 5	—	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—	—
End-to-end VLA / WAM policies						
$\pi_{0.5}$	—	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—	—
EventVLA	—	—	—	—	—	—
X-VLA	—	—	—	—	—	—
DM0.5	—	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—	—
OpenWAM- α	—	—	—	—	—	—
X-WAM	—	—	—	—	—	—
Fast-WAM	—	—	—	—	—	—

Table 7: Motor Core success rates. All measurement cells are *not evaluated*.

Model / system	FR1	FR2	PR1	PR2	PR3	Mean
Coding agents + proposed Inspect EEf adapter						
GPT-6 Astra	—	—	—	—	—	—
Claude Opus 5	—	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—	—
End-to-end VLA / WAM policies						
$\pi_{0.5}$	—	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—	—
EventVLA	—	—	—	—	—	—
X-VLA	—	—	—	—	—	—
DM0.5	—	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—	—
OpenWAM- α	—	—	—	—	—	—
X-WAM	—	—	—	—	—	—
Fast-WAM	—	—	—	—	—	—

Table 8: Coupling success rates, CP01–CP06. All measurement cells are *not evaluated*.

Model / system	CP01	CP02	CP03	CP04	CP05	CP06
Coding agents + proposed Inspect EEf adapter						
GPT-6 Astra	—	—	—	—	—	—
Claude Opus 5	—	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—	—
End-to-end VLA / WAM policies						
$\pi_{0.5}$	—	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—	—
EventVLA	—	—	—	—	—	—
X-VLA	—	—	—	—	—	—
DM0.5	—	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—	—
OpenWAM- α	—	—	—	—	—	—
X-WAM	—	—	—	—	—	—
Fast-WAM	—	—	—	—	—	—

Table 9: Coupling success rates, CP07–CP12. All measurement cells are *not evaluated*.

Model / system	CP07	CP08	CP09	CP10	CP11	CP12
Coding agents + proposed Inspect EEf adapter						
GPT-6 Astra	—	—	—	—	—	—
Claude Opus 5	—	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—	—
End-to-end VLA / WAM policies						
$\pi_{0.5}$	—	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—	—
EventVLA	—	—	—	—	—	—
X-VLA	—	—	—	—	—	—
DM0.5	—	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—	—
OpenWAM- α	—	—	—	—	—	—
X-WAM	—	—	—	—	—	—
Fast-WAM	—	—	—	—	—	—

Table 10: Reserved behavioral diagnostics. Applicability and denominators must be declared; no internal state is assumed observable. All measurement cells are *not evaluated*.

Model / system	Premature advance	State mismatch	Wrong binding	Missed window	Recovery cost
Coding agents + proposed Inspect EEf adapter					
GPT-6 Astra	—	—	—	—	—
Claude Opus 5	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—
End-to-end VLA / WAM policies					
$\pi_{0.5}$	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—
EventVLA	—	—	—	—	—
X-VLA	—	—	—	—	—
DM0.5	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—
OpenWAM- α	—	—	—	—	—
X-WAM	—	—	—	—	—
Fast-WAM	—	—	—	—	—

Table 11: Reserved runtime measurements. Report a separate table for each clock/sensor condition. Decision latency is in milliseconds; update rate counts new accepted commands, not interpolated ticks. All measurement cells are *not evaluated*.

Model / system	Latency p50/p95	Update Hz	Deadline miss	Episode seconds	API calls	API cost USD
Coding agents + proposed Inspect EEf adapter						
GPT-6 Astra	—	—	—	—	—	—
Claude Opus 5	—	—	—	—	—	—
GPT-5.6 Sol	—	—	—	—	—	—
DeepSeek-V4.1-Flash	—	—	—	—	—	—
Gemini 3.7 Flash	—	—	—	—	—	—
GLM-5.3 Flash	—	—	—	—	—	—
Claude Opus 4.8	—	—	—	—	—	—
GPT-5.6 Luna	—	—	—	—	—	—
GPT-5.6 Terra	—	—	—	—	—	—
End-to-end VLA / WAM policies						
$\pi_{0.5}$	—	—	—	—	—	—
OpenVLA-OFT	—	—	—	—	—	—
GR00T-N1.7	—	—	—	—	—	—
Hy-Embodied-0.5-VLA	—	—	—	—	—	—
EventVLA	—	—	—	—	—	—
X-VLA	—	—	—	—	—	—
DM0.5	—	—	—	—	—	—
GalaxeaVLA G0.5	—	—	—	—	—	—
Xiaomi-Robotics-1	—	—	—	—	—	—
OpenWAM- α	—	—	—	—	—	—
X-WAM	—	—	—	—	—	—
Fast-WAM	—	—	—	—	—	—

D. Source Provenance and Pending Configuration

The authoritative local inputs are the 17 September 2026 MEMbench text snapshot, the twelve-case coupling JSON, and the corresponding coupling protocol. The earlier nine-case cognitive drafts, three-case coupling proposal and 42-family expansion are not used to redefine the task count. The source archive retains the Chinese originals so wording, parameter values and proposal status can be checked against the English report.

External model identities come from RLE-Bench’s public model/runtime data and RoboDojo’s GPT-6-Astra comparison report. The latter’s public-policy comparison identifies a 10 September 2026 snapshot, whereas the original RoboDojo paper describes an earlier thirty-policy simulation evaluation. We do not treat those two snapshots as the same leaderboard. The XPolicyLab and Inspect Robots documentation support the interface discussion, not a claim that PhysCo has already integrated them. Public webpages are cited as webpages or software resources; RLE-Bench is not assigned an invented arXiv identifier.

Table 12: Run-configuration inventory. Pending fields are intentionally unfilled.

Model / system	Runtime / serving	Model ID	History	Budget
GPT-6 Astra	Codex	—	—	—
Claude Opus 5	Claude Code	—	—	—
GPT-5.6 Sol	Codex	—	—	—
DeepSeek-V4.1-Flash	Claude Code	—	—	—
Gemini 3.7 Flash	Antigravity CLI	—	—	—
GLM-5.3 Flash	Claude Code	—	—	—
Claude Opus 4.8	Claude Code	—	—	—
GPT-5.6 Luna	Codex	—	—	—
GPT-5.6 Terra	Codex	—	—	—
$\pi_{0.5}$	XPolicyLab	—	—	—
OpenVLA-OFT	XPolicyLab	—	—	—
GR00T-N1.7	XPolicyLab	—	—	—
Hy-Embodied-0.5-VLA	XPolicyLab	—	—	—
EventVLA	XPolicyLab	—	—	—
X-VLA	XPolicyLab	—	—	—
DM0.5	XPolicyLab	—	—	—
GalaxeaVLA G0.5	XPolicyLab	—	—	—
Xiaomi-Robotics-1	XPolicyLab	—	—	—
OpenWAM- α	XPolicyLab	—	—	—
X-WAM	XPolicyLab	—	—	—
Fast-WAM	XPolicyLab	—	—	—

Table 13: Additional policy references from the archived RoboDojo comparison. These are source roster entries, not extra PhysCo result rows. They include non-VLA policy baselines and preserve source naming. No integration or evaluation is implied.

Additional source entry	Additional source entry
Meituan-Robotics-0	ABot-M0
Spatial Forcing	Pi-0
InternVLA-A1.5	InternVLA-A1
StarVLA-PI_v3	SmolVLA (Single Task)
VLAct	LDA-1B
MolmoAct2	ACT (Single Task)
StarVLA-GR00T	GO-1
StarVLA-OFT	H-RDT
Xiaomi-Robotics-0	RDT
StarVLA	DM0
GigaWorld-Policy-0	Dexora-1B
GalaxeaVLA (G0)	A1
LingBot-VLA	Spirit v1.5
AHA-WAM	TinyVLA

The technical-report style is adapted from RoboDojo’s `dojo_report.sty`. Its source is preserved with attribution; the PhysCo version removes original author branding, logos and the hard-coded publication date. All figures in this report are newly generated schematic diagrams. Their layout draws on existing local PhysCo overview illustrations, but their model

Table 14: Configuration items to freeze before evaluation. Pending settings are not silently assigned values from another benchmark.

Item	Required record
Task realization	Backend/version, assets and licenses, robot embodiment, camera calibration and permitted sensors.
Task difficulty	Missing E/P/S/FR1/FR2 tables; numeric PR2/PR3 thresholds; calibrated CP01–CP12 parameters.
Action interface	Inspect EEF tool schemas and bounds; IK/interpolation; VLA action mode, units and chunk duration.
Observations and history	Camera ordering and frame rate, proprioception dimensions, history retention, allowed files and tool access.
Clock and waiting	Continuous or paused inference, timestamp synchronization, command queues and no-new-action hold behavior.
Model identity	Provider/API version or checkpoint hash, runtime version, inference parameters and training/adaptation access.
Evaluation schedule	Difficulty weights, episode count, random seeds, initial-state/event distributions and held-out conditions.
Resource budgets	Environment duration, control steps, tool/model calls, retry/probe limits and any development budget.
Scoring and diagnostics	Versioned success/checkpoint predicates, logging visibility, applicable diagnostics and missing-data handling.
Uncertainty	Independent run unit, repeat structure, confidence-interval procedure and coverage requirements.

paths, task counts and labels follow this draft. No source figure’s demonstration score or timing annotation is reproduced as a PhysCo observation.

The package includes modular section and table files, English task metadata, the empty numerical result template, FigureSpec sources, and build instructions. Preserved external source snapshots may contain *external* experiment scores; those files are research evidence only and are never used to populate the PhysCo tables.